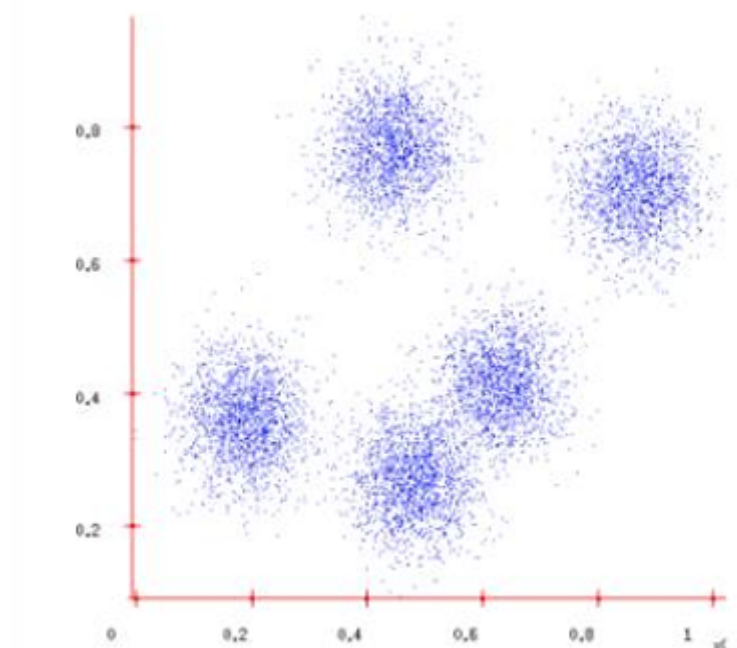


- [1. 聚类 \(Clustering\)](#)
- [2. K-means算法](#)
 - [2.1 交替最小化 \(Alternating Minimization\)](#)
 - [2.2 K-means算法](#)
 - [2.2.1 K-means算法的数学表达](#)
 - [2.2.2 应用](#)
 - [2.2.2.1 向量量化 \(Vector Quantization\)](#)
 - [2.2.2.2 图像分割](#)
 - [2.2.3 相关问题](#)
 - [2.2.3.1 K-means 算法的收敛性](#)
 - [2.2.3.1.1 陷入局部最小值 \(Local Minima\)](#)
 - [2.2.3.2 选择最佳的簇数 K](#)
 - [2.2.3.3 选择合适的距离度量](#)
 - [2.2.3.4 选择不同的距离度量](#)
 - [2.2.3.5 簇中心 \$\mu_k\$ 为什么是已分配点的平均值](#)
 - [2.3 软 K-means \(Soft K-means\)](#)
 - [2.3.1 软 K-means的数学表达](#)
 - [2.3.2 软 K-means 剩余问题](#)
 - [2.4 概率生成模型](#)
 - [2.4.1 单变量高斯分布 \(Univariate Gaussian distribution\)](#)
 - [2.4.2 多变量高斯分布 \(Multivariate Gaussian distribution\)](#)
 - [2.4.3 生成模型 \(The Generative Model\)](#)
 - [2.4.4 最大似然估计](#)
 - [2.4.5 高斯混合模型 \(GMM\)](#)
 - [2.4.5.1 使用最大似然估计 \(Maximum Likelihood Estimation, MLE\) 来拟合高斯混合模型 \(Gaussian Mixture Model, GMM\)](#)
 - [2.4.5.2 EM算法](#)
 - [2.4.5.2.1 EM算法的数学表达](#)
 - [2.4.5.2.2 EM算法的小结](#)
 - [2.4.5.2.3 K-Means 和 EM 算法 的对比](#)

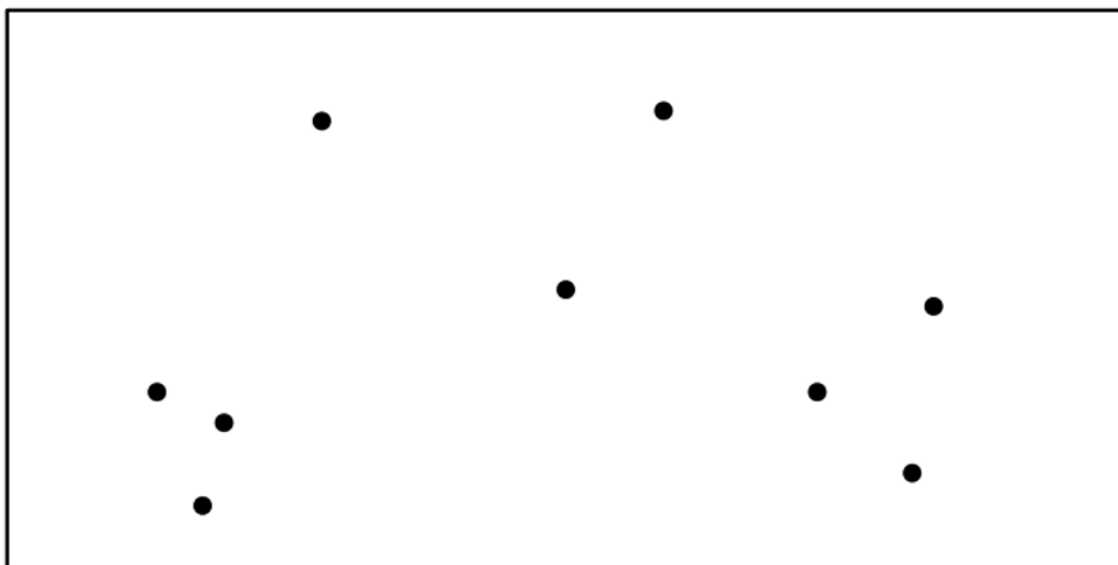
1. 聚类 (Clustering)

数据有时会形成簇，即数据点在簇内彼此相似，而不同簇之间的数据点则不相似。



将数据点分组到簇中，而没有观察到的标签，这个过程称为聚类。这是一种无监督学习技术，因为数据点没有预先标记的类别标签。

例如，可以根据主题对机器学习论文进行聚类（使用深度学习、贝叶斯模型等方法）。但主题本身是未观察到的（即无监督的）。



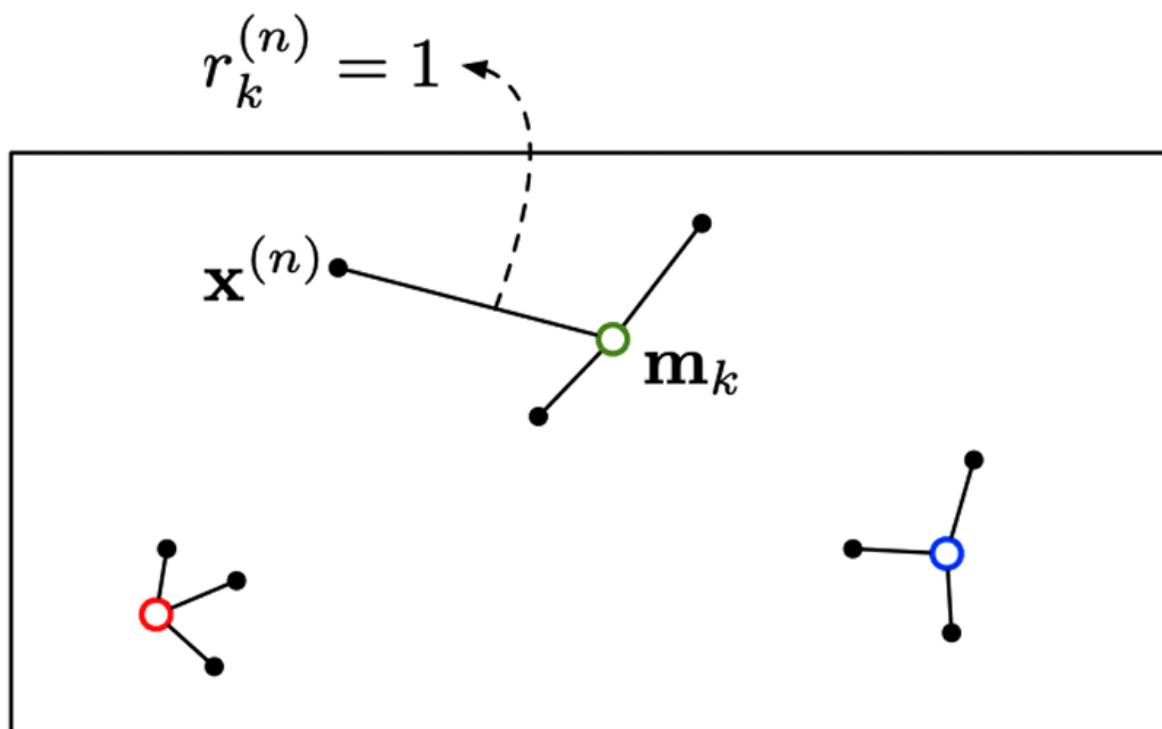
假设数据集 $\{x^{(1)}, \dots, x^{(N)}\}$ 存在于一个欧几里得空间中，其中每个数据点 $x^{(n)}$ 属于 R^D 即 D 维实数空间。

假设每个数据点属于 K 个簇中的一个。这意味着数据点可以被分为 K 个不同的组，每个组代表一个簇。那么同一簇中的数据点是相似的，即它们在欧几里得距离上彼此接近。

那么现在我们到底如何设计一个算法以识别这些簇（即确定哪些数据点属于每个簇）呢？或者我们将这个问题具体为一个优化问题，这样我们就可以找到一个方法来最小化或最大化某个目标函数，以确定最佳的簇划分。

2. K-means算法

K-means 算法的目标是找到簇中心 $\{m_k\}_{k=1}^K$ 和数据点的分配 $\{r^{(n)}\}_{n=1}^N$ ，以最小化数据点 $\{m_k\}_{k=1}^K$ 到其分配的簇中心的平方距离之和。



数据样本 $n = 1, \dots, N$: 每个数据点 $x^{(n)}$ 属于 R^D (这是可以被观察到的, 换句话说我们是可以直接从数据集中得到的)。

簇中心 $k = 1, \dots, K$: 每个簇中心 m_k 也属于 R^D (未观察到的, 换句话说这些事需要分析或者计算才能得到的)。

每个数据点 n 的簇分配 $r^{(n)}$ 是一个 R^K 的独热编码 (未观察到的)。独热编码表示每个数据点属于哪个簇, 例如, 如果数据点属于簇 k , 则 $r_k^{(n)} = 1$, 其他位置为 0。

数学表达式为: $\min_{\{m_k\}, \{r^{(n)}\}} J(\{m_k\}, \{r^{(n)}\}) = \min_{\{m_k\}, \{r^{(n)}\}} \sum_{n=1}^N \sum_{k=1}^K r_k^{(n)} \|m_k - x^{(n)}\|^2$

其中, $r_k^{(n)}$ 是一个指示函数, 如果数据点 $x^{(n)}$ 被分配到簇 k , 则 $r_k^{(n)} = 1$, 否则为 0。(例如, 如果 $x^{(n)}$ 被分配到簇 $k = 3$, 则 $r^{(n)} = [0, 0, 1, 0, \dots]$) $\|m_k - x^{(n)}\|^2$ 是数据点 $x^{(n)}$ 到簇中心 m_k 的欧几里得距离的平方。

而这个寻找最优解是一个 NP-hard 问题。

我们回到这里, 数据点 $x^{(n)}$ 被分配到簇 k , 则 $r_k^{(n)} = 1$, 否则为 0。(例如, 如果 $x^{(n)}$ 被分配到簇 $k = 3$, 则 $r^{(n)} = [0, 0, 1, 0, \dots]$)。

这里内层求和是对 K 项进行的, 但只有一个项是非零的, 即数据点被分配到的簇对应的项。

如果 $x^{(n)}$ 被分配到簇 $k = 3$, 则 $r^{(n)} = [0, 0, 1, 0, \dots]$ 这个例子就可以变为

$\sum_{k=1}^K r_k^{(n)} \|m_k - x^{(n)}\|^2 = \|m_3 - x^{(n)}\|^2$ 。也就是说对于每个数据点, 我们只计算它到其分配的簇中心的距离。

2.1 交替最小化 (Alternating Minimization)

我们回到这个优化问题上, $r^{(n)}\} \sum_{n=1}^N \sum_{k=1}^K r_k^{(n)} \|m_k - x^{(n)}\|^2$ 。

问题的难点在于同时最小化簇中心 $\{m_k\}$ 和簇分配 $\{r^{(n)}\}$ 。但是如果固定其中一个变量 (簇中心或簇分配), 然后对另一个变量进行最小化, 那么问题就变得容易解决。

所以现在我们尝试固定簇中心 $\{m_k\}$, 那么我们现在可以很容易地找到每个样本 n 的最优分配 $\{r^{(n)}\}$, 所以现在我们得到的优化问题为:

$\min_{\{r^{(n)}\}} \sum_{k=1}^K r_k^{(n)} \|m_k - x^{(n)}\|^2$

我们现在将每个数据点 $x^{(n)}$ 通过计算它到每个簇中心 m_k 的距离, 将其分配到最近的簇中心以找到最优

分配。

$$r_k^{(n)} = \begin{cases} 1 & \text{if } k = \arg \min_j \|x^{(n)} - m_j\|^2 \\ 0 & \text{otherwise} \end{cases}$$

因此就是如果 $x^{(n)}$ 被分配到簇 $\hat{k}$ ，则 $r^{(n)}$ 是一个 K -维向量，其中只有第 $\hat{k}$ 个元素为1，其余元素为0。

即：

$$r^{(n)} = [0, 0, \dots, 1, \dots, 0]^\top$$

所以现在我们交替地优化两个变量——簇中心 $\{m_k\}$ 和数据点的簇分配 $\{r^{(n)}\}$ 。

对于每个簇 l ($l = 1, 2, \dots, K$)，我们将簇中心设置为其分配的数据点的平均值。数学上，这可以通过以下步骤实现：

首先，对目标函数关于簇中心 m_l 求导，并令导数等于0来找到最优解：

$$0 = \frac{\partial}{\partial m_l} \sum_{n=1}^N \sum_{k=1}^K r_k^{(n)} \|m_k - x^{(n)}\|^2$$

$$\text{简化后得到： } 0 = 2 \sum_{n=1}^N r_l^{(n)} (m_l - x^{(n)})$$

$$\text{从而得到簇中心 } m_l \text{ 的更新公式： } m_l = \frac{\sum_n r_l^{(n)} x^{(n)}}{\sum_n r_l^{(n)}}$$

所以交替最小化 (alternating minimization) 的过程为：

我们交替地最小化目标函数 $J(\{m_k\}, \{r^{(n)}\})$ 关于簇中心 $\{m_k\}$ 和簇分配 $\{r^{(n)}\}$ 。

这个过程通常从一个初始的簇中心和簇分配开始，然后交替更新这两个变量，直到收敛到一个稳定的解。

2.2 K-means算法

K-means算法就是一种交替最小化的体现。

从宏观上来看，K-means包含两部分：

1. 初始化：

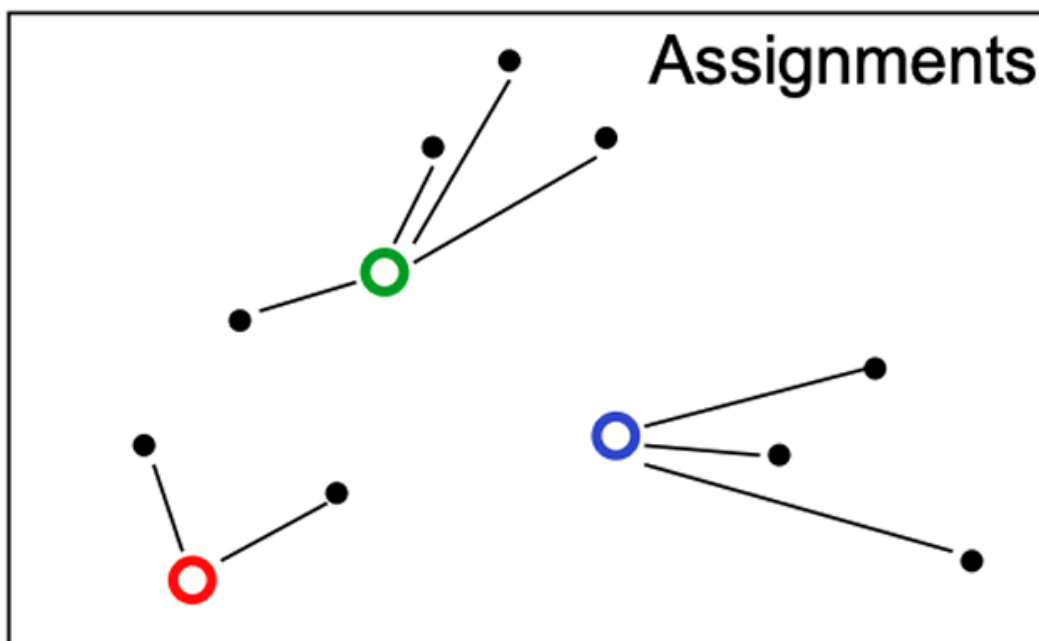
随机初始化簇中心。

2. 迭代过程：

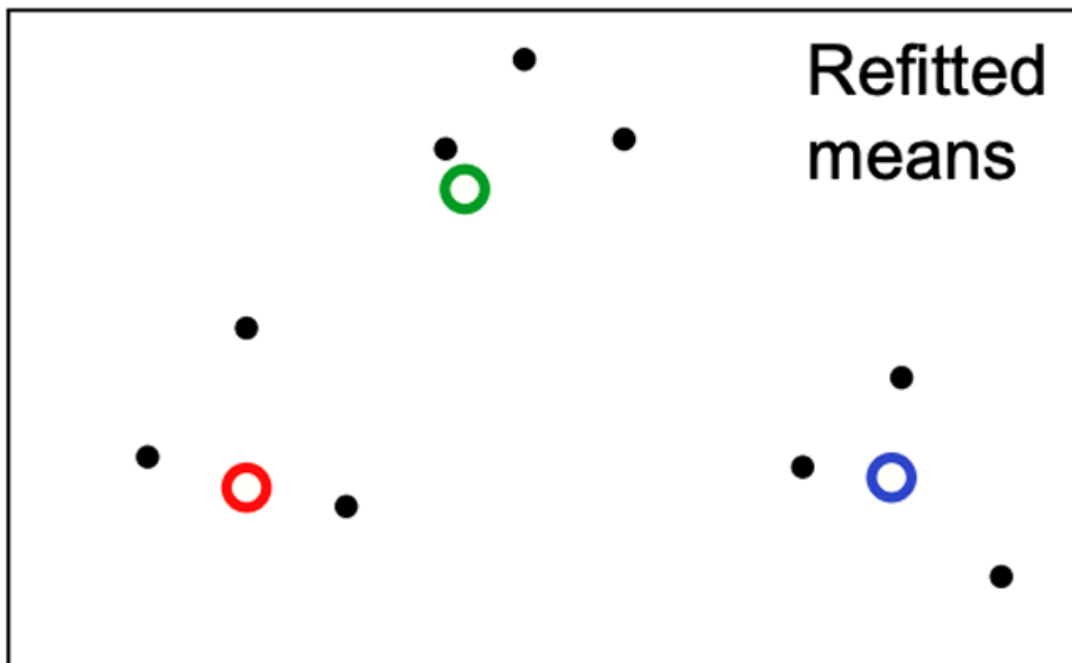
算法在两个主要步骤之间交替进行，直到满足停止条件（例如，簇中心不再显著变化或达到最大迭代次数）。

• 分配步骤 (Assignment step)：

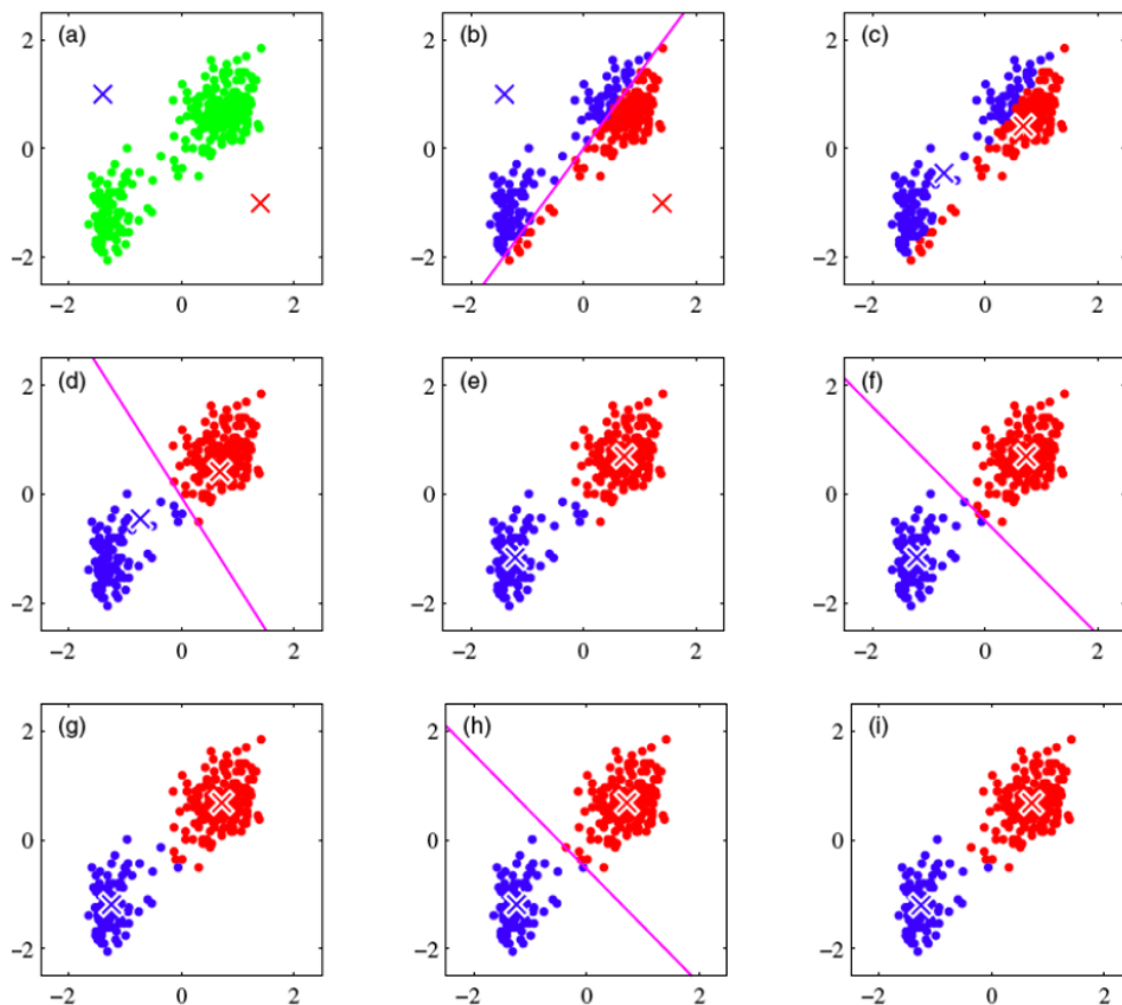
将每个数据点分配给最近的簇中心。这是通过计算每个数据点到各个簇中心的距离，并将其分配给距离最近的簇中心来实现的。



- 拟合步骤 (Refitting step) :
移动每个簇中心到分配给它的数据点的均值位置。这意味着计算每个簇中所有数据点的平均值, 并更新簇中心到这个平均位置。



下张图更清晰展示了这个过程, 不断将每个数据点分配给最近的簇中心并且更新簇中心, 两个步骤交替进行。



2.2.1 K-means算法的数学表达

我们现在看一下这些操作的数学表达。

1. 初始化:

随机设置 K 个簇中心 $\mathbf{m}_1, \dots, \mathbf{m}_K$ 为任意值。

2. 迭代过程:

算法在两个主要步骤之间交替进行，直到簇分配不再发生变化，即算法收敛。

- 分配步骤 (Assignment step) 是在优化目标函数 J 关于簇分配 $\{r^{(n)}\}$ 。：
将每个数据点 $\mathbf{x}^{(n)}$ 分配给最近的簇中心。这通过计算每个数据点到所有簇中心的距离，并选择距离最小的簇中心来实现。数学表达式为： $\hat{k}^{(n)} = \arg \min_k \|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2$
使用 1-of- K 编码表示簇分配：
$$r_k^{(n)} = \mathbb{I}[\hat{k}^{(n)} = k] \quad \text{for } k = 1, \dots, K$$
- 拟合步骤 (Refitting step) 是在优化目标函数 J 关于簇中心 $\{\mathbf{m}_k\}$ ：
将每个簇中心设置为分配给它的数据点的均值。这通过计算每个簇中所有数据点的平均值来实现。
数学表达式为：
$$\mathbf{m}_k = \frac{\sum_n r_k^{(n)} \mathbf{x}^{(n)}}{\sum_n r_k^{(n)}}$$

2.2.2 应用

2.2.2.1 向量量化 (Vector Quantization)

我们前面提到过向量量化是一种数据压缩技术，这里它通过减少图像中颜色的数量来降低图像的存储空间需求，同时尽量保持图像的视觉质量。

这里具体操作如下：

1. 构建“数据集”:

给定一张图像，首先将其构建为一个“数据集”，其中每个“数据点”是一个像素，由其 RGB 颜色强度表示。这意味着每个像素可以被看作是三维空间中的一个点，其坐标是红、绿、蓝三个颜色通道的强度值。

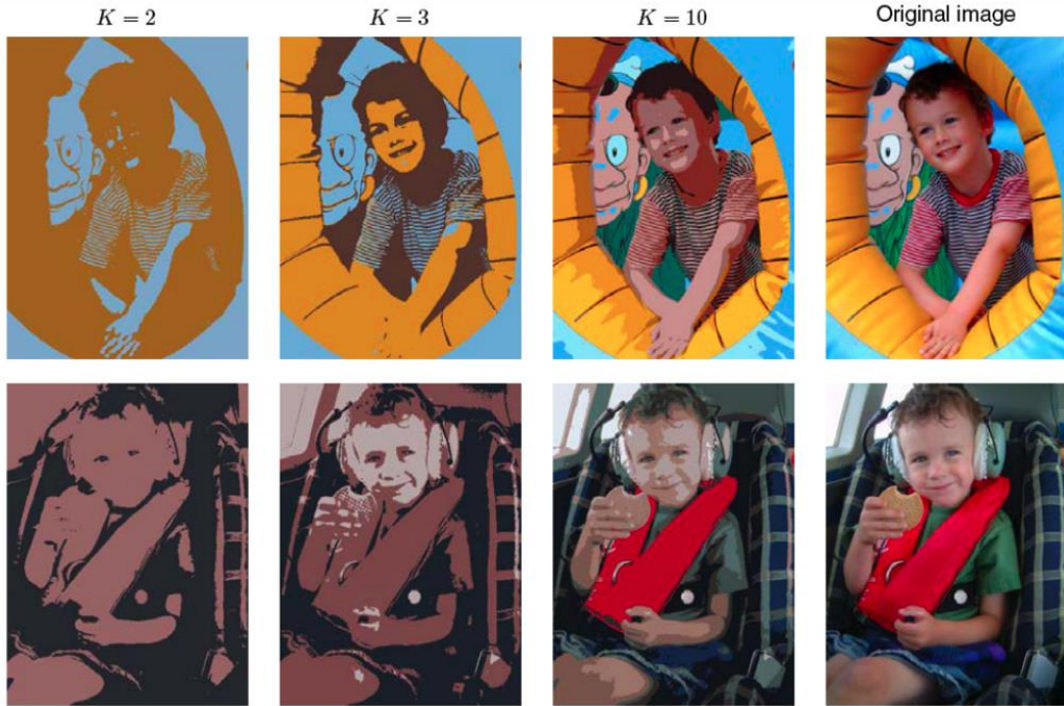
2. 运行 K-means 算法:

在这个“数据集”上运行 K-means 算法，将像素点划分为 K 个簇。每个簇由一个中心点（簇中心）表示，这个中心点是该簇中所有像素点的“平均”颜色。

3. 替换每个像素:

将图像中的每个像素替换为其所属簇的中心颜色。这意味着每个像素的颜色将被量化为最接近它的

簇中心的颜色。



2.2.2.2 图像分割

我们可以使用 K-means 算法进行图像分割，可以将图像分割成具有相似颜色和纹理的超像素，这些超像素可以用于进一步的图像分析和处理任务，如目标识别、图像压缩等。

超像素是图像处理中的一种技术，用于将图像分割成具有相似颜色和纹理的较大块，这些块通常比单个像素具有更多的语义信息。

具体操作如下：

1. 构建“数据集”：

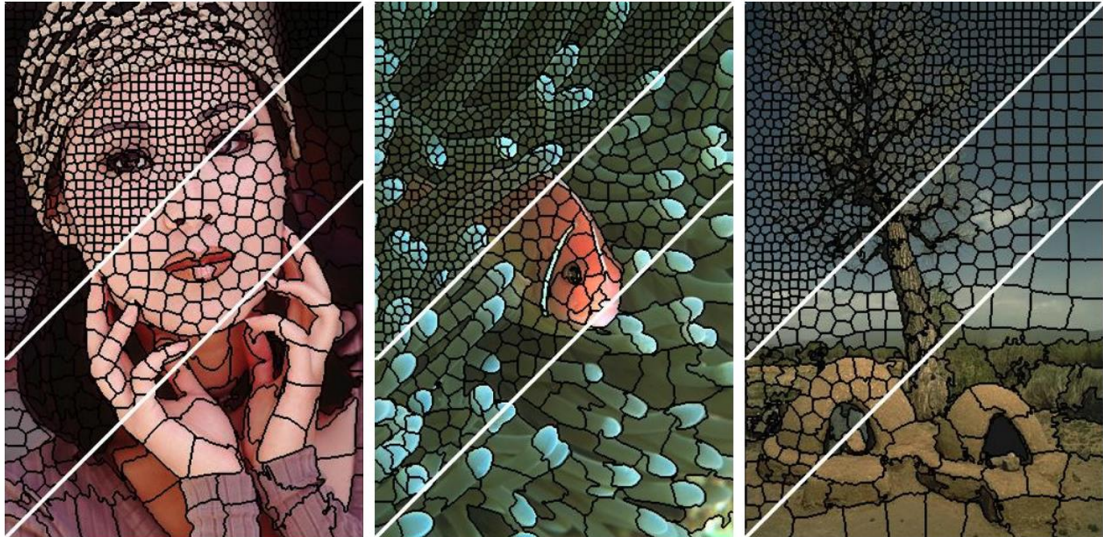
给定一张图像，首先将其构建为一个“数据集”，其中每个“数据点”是一个像素，由其 RGB 颜色强度和它在图像中的位置（网格位置）表示。这意味着每个像素可以被看作是更高维空间中的一个点，其坐标包括颜色通道的强度值和位置信息。

2. 运行 K-means 算法：

在这个扩展的“数据集”上运行 K-means 算法，将像素点划分为 K 个簇。每个簇由一个中心点（簇中心）表示，这个中心点是该簇中所有像素点的颜色和位置的“平均”值。

3. 生成超像素：

将图像中的每个像素替换为其所属簇的中心颜色和位置。这意味着每个簇中的像素将被合并为一个具有统一颜色和纹理的较大块，即超像素。



2.2.3 相关问题

我们现在有一些细节的问题没有解决，后面将逐步介绍这些。这些问题有：

1. K-means算法会收敛吗？
2. 如何选择最佳的簇数 K ？
3. 我们如何选择合适的距离度量？
4. 如果我们使用不同的距离度量会怎么样？
5. 为什么要将簇中心 m_k 更新为已分配点的平均值？

2.2.3.1 K-means 算法的收敛性

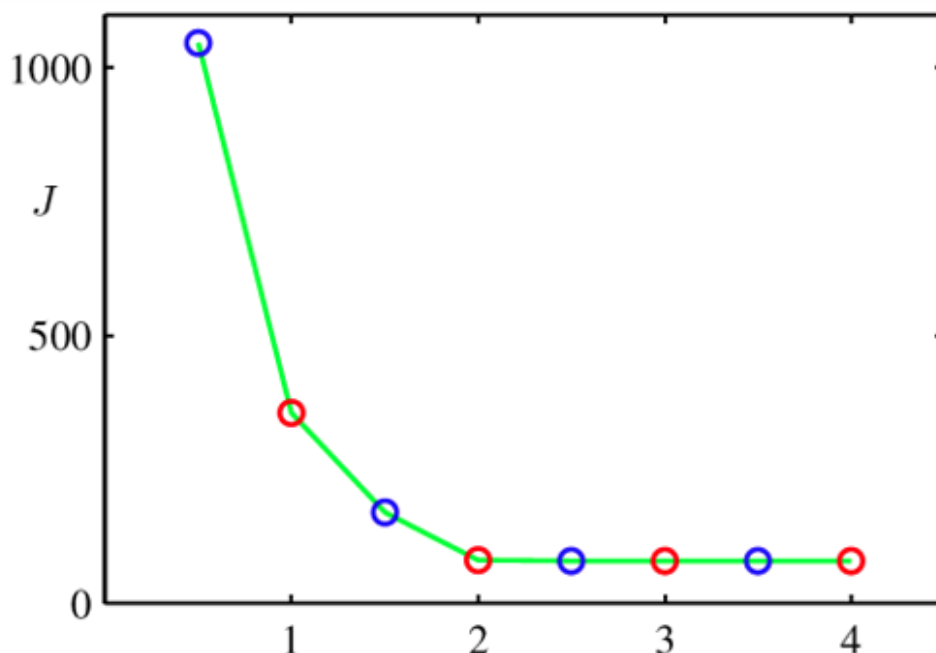
K-means 算法在每次迭代中都会减少目标函数 J 的值，即数据点到其分配的簇中心的平方距离之和。

当簇分配发生变化时，即数据点被重新分配到更近的簇中心， J 会减少。

当簇中心移动时，即簇中心更新为分配给它的数据点的均值， J 也会减少。

算法通过检查簇分配是否在分配步骤中发生变化来测试收敛性。如果在一次分配步骤中没有数据点的簇分配发生变化，那么算法已经收敛，至少达到了一个局部最小值。

由于可能的簇分配数量是有限的，K-means 算法总会在有限的迭代次数后收敛。这是因为每次迭代都会改变一些数据点的簇分配，而总的簇分配可能性是有限的。



图中展示了 K-means 成本函数在每次分配步骤（蓝色）和拟合步骤（红色）后的变化。可以看到，成本函数在前几次迭代中迅速下降，并在第三次拟合步骤后趋于稳定，表明算法已经收敛。

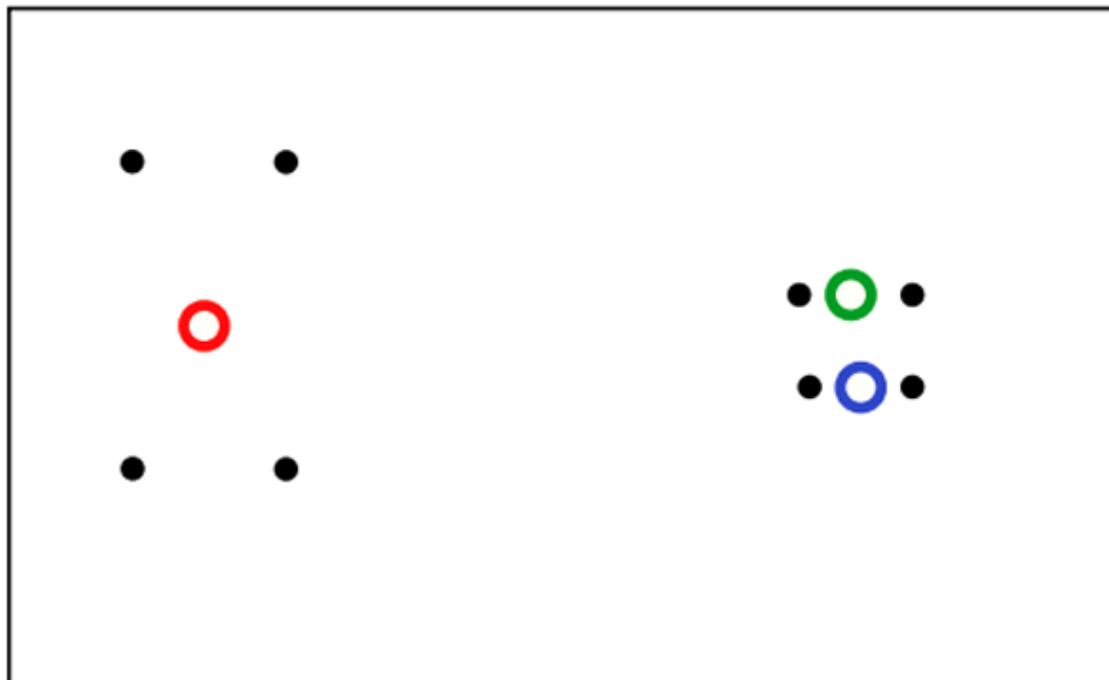
2.2.3.1.1 陷入局部最小值 (Local Minima)

因此这就延伸到了一个问题——陷入局部最小值 (Local Minima)。

K-means 算法的目标函数 J （即数据点到其分配的簇中心的平方距离之和）是非凸的。这意味着目标函数的等高线可能包含多个凹陷区域，算法可能会收敛到这些凹陷区域中的一个，而不是全局最小值。没有确定的方法可以完全防止 K-means 算法陷入局部最小值。然而，可以通过尝试多种不同的初始簇中心来增加找到更好解的机会。

一种常见的策略是多次运行 K-means 算法，每次使用不同的随机初始簇中心。然后，选择产生最低目标函数值的解作为最终结果。这种方法可以提高找到全局最小值的概率，尽管不能保证一定找到。

如下图所示，就是一个局部最小值。



如果右边分为一个簇，左边分为两个簇的结果会比现在图中所示的更好。

2.2.3.2 选择最佳的簇数 K

选择最佳的簇数 K 是一个挑战，因为没有通用的方法适用于所有数据集。一些常用的方法包括肘部法则 (Elbow Method)、轮廓系数 (Silhouette Coefficient) 和间隙统计 (Gap Statistic)。这些方法试图找到一个“肘点”，即增加簇数对聚类质量的改进开始显著减小的点。

2.2.3.3 选择合适的距离度量

选择距离度量通常取决于数据的特性和聚类的目标。例如，如果数据是文本，可能会使用余弦相似度；如果数据具有不同的量纲，可能会使用标准化的欧几里得距离或马氏距离。此外，可以尝试多种距离度量，并根据聚类结果的质量（如轮廓系数、Davies-Bouldin 指数等）来选择最佳的距离度量。

2.2.3.4 选择不同的距离度量

这个问题如上一个问题类似。

K-means 算法默认使用欧几里得距离来计算数据点之间的距离。然而，也可以使用其他距离度量，如曼哈顿距离、余弦相似度或马氏距离等。不同的距离度量可能会导致不同的聚类结果，因为它们强调数据点之间的不同方面。例如，余弦相似度适用于文本数据，而曼哈顿距离可能在处理具有不同量纲的特征时更有用。

2.2.3.5 簇中心 m_k 为什么是已分配点的平均值

这样做的目的是为了最小化簇内数据点到簇中心的距离，这是 K-means 算法的目标函数。通过将簇中心设置为数据点的平均值，可以确保簇中心尽可能地接近它所代表的所有数据点，从而减少目标函数的值。

2.3 软 K-means (Soft K-means)

软分配 (Soft Assignments) 与传统的硬分配 (每个数据点严格属于一个簇) 不同，软分配允许数据点以一定的概率属于多个簇。例如，一个数据点可能以 70% 的责任度属于一个簇，同时以 30% 的责任度属于另一个簇。

软分配允许簇在拟合步骤中使用更多关于数据的信息。这是因为每个数据点对所有簇都有贡献，而不仅是对一个簇有贡献。

软分配的决策通常基于数据点到各个簇中心的距离。数据点更可能被分配到距离较近的簇，但也会考虑到其他簇。

在多类分类问题中，我们通常使用 softmax 函数来计算一个样本属于每个类别的概率。这与软 K-means 中的软分配类似，都是基于距离或相似度来分配概率。

因此在传统的 K-means 中，我们使用 1-of-K 编码来表示簇分配，即每个数据点的分配是一个独热向量，只有一个位置是 1，其余都是 0。

而在软 K-means 中，我们使用 softmax 函数来计算每个数据点属于每个簇的概率，得到的是一个概率分布，其中所有概率之和为 1。

2.3.1 软 K-means 的数学表达

1. 初始化:

随机设置 K 个簇中心 $\mathbf{m}_k$ 。

2. 重复直到收敛:

算法在两个主要步骤之间交替进行，直到满足停止条件（例如，目标函数 J 的变化非常小）。

• 分配步骤 (Assignment step) :

每个数据点 n 被赋予一个“软分配”给每个簇中心 k ，基于它们的责任度 (responsibilities)。数学表达式为: $r_k^{(n)} = \frac{\exp[-\beta \|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2]}{\sum_j \exp[-\beta \|\mathbf{m}_j - \mathbf{x}^{(n)}\|^2]}$ 其中, β 是一个参数, 控制数据点分配的“软”程度。 $r^{(n)}$ 是通过 softmax 函数计算得到的, 确保所有簇的责任度之和为 1

• 拟合步骤 (Refitting step) :

模型参数, 即簇中心, 被调整以匹配它们所负责的数据点的样本均值。数学表达式为:

$$\mathbf{m}_k = \frac{\sum_n r_k^{(n)} \mathbf{x}^{(n)}}{\sum_n r_k^{(n)}}$$

这里, 簇中心 $\mathbf{m}_k$ 是根据分配给该簇的所有数据点的加权平均来更新的, 权重是每个数据点的责任度。

2.3.2 软 K-means 剩余问题

1. 如何设置参数 β ?

- 参数 β 控制数据点分配的“软”程度。较大的 β 值会使分配更加“硬”, 即数据点更可能被严格地分配给一个簇; 较小的 β 值会使分配更加“软”, 即数据点可能以一定的概率属于多个簇。
- 选择合适的 β 值可能需要一些实验和调整, 因为它会影响聚类结果的质量和解释性。

2. 簇具有不等的权重和宽度?

- 在某些情况下, 数据可能自然地形成具有不同权重 (即数据点数量) 和宽度 (即簇的扩散程度) 的簇。传统的 K-means 算法可能难以处理这种情况, 因为它假设所有簇具有相同的形状和大小。
- 软 K-means 算法通过引入概率分配, 可以更好地处理具有不同权重和宽度的簇, 但仍然可能需要进一步的方法来优化簇的形状和大小。

这些问题都可以通过使用生成模型重新构建聚类问题。生成模型是一种统计模型, 它假设数据是由一个潜在的生成过程产生的。在聚类问题中, 生成模型可以假设数据是由多个潜在的分布 (如高斯分布) 生成的, 每个分布对应一个簇。通过使用生成模型, 我们可以更灵活地建模簇的形状、大小和权重。此外, 生成模型通常具有更好的理论基础和数学性质, 可以提供更稳定和可靠的聚类结果。

2.4 概率生成模型

聚类问题可以通过概率生成模型来重新表述。

一个显而易见的方法是想象数据是由一个生成模型产生的。这个生成模型可以是任何能够产生类似观测数据的概率模型, 例如高斯混合模型 (Gaussian Mixture Model, GMM)。

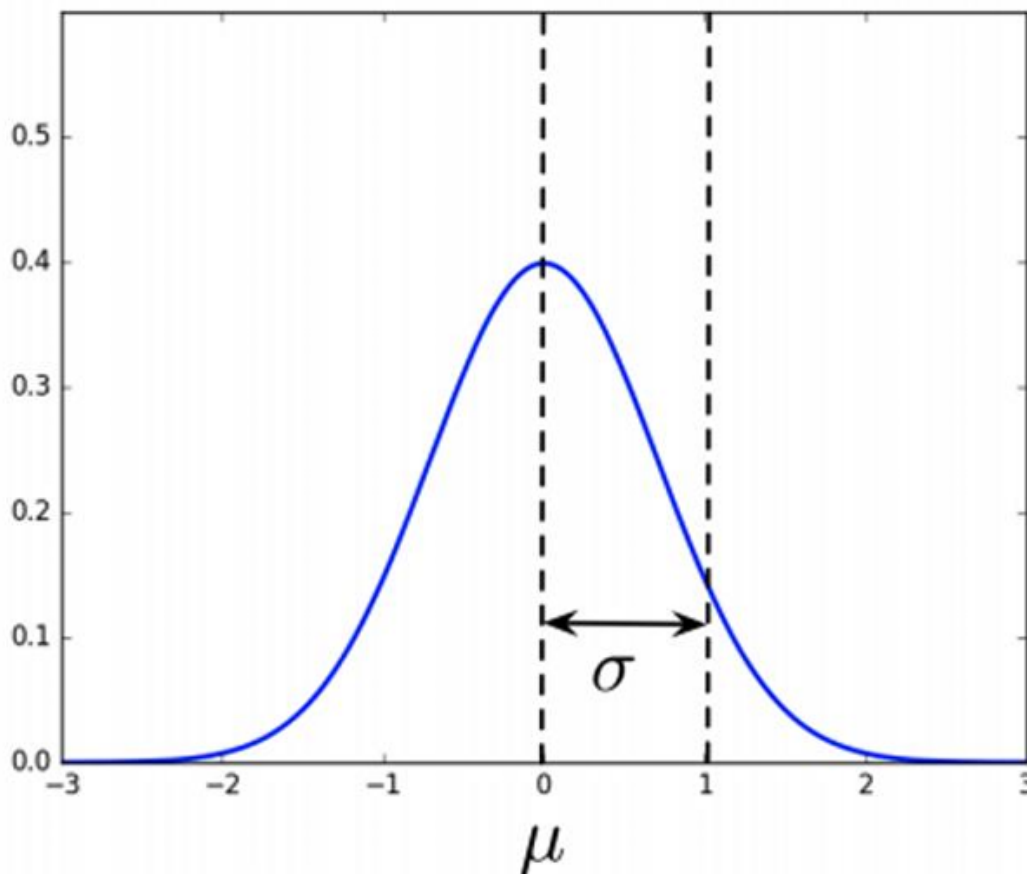
我们通过最大似然估计 (Maximum Likelihood Estimation, MLE) 来调整模型参数。

大似然估计是一种统计方法，用于估计模型参数，使得在给定参数下观测到当前数据的概率最大化。因此现在在聚类问题中，我们希望找到一组模型参数，使得在这些参数下生成我们观测到的数据的概率最大化。换句话说，我们希望找到一组参数，使得模型最有可能产生我们实际观测到的数据。

2.4.1 单变量高斯分布 (Univariate Gaussian distribution)

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

其中， μ 是分布的均值， σ^2 是方差， σ 是标准差。



上图展示了高斯分布的典型形状，它是一个钟形曲线，均值 μ 决定了曲线的中心，标准差 σ 决定了曲线的宽度。

中心极限定理指出，大量独立随机变量的和近似服从高斯分布。这一定理在统计学中非常重要，因为它意味着在许多情况下，即使原始数据不是正态分布的，它们的和或平均值也可能是近似正态分布的。在机器学习中，高斯分布被广泛使用，因为它们使得计算变得简单。许多机器学习算法，如线性回归、逻辑回归和神经网络中的某些层，都假设数据或误差项服从高斯分布，或者至少近似服从高斯分布。

2.4.2 多变量高斯分布 (Multivariate Gaussian distribution)

多变量数据涉及多个测量值，这些测量值可以来自不同的传感器或不同的特征。

数据集有 D 个特征或属性，每个特征都是数据的一个维度。

数据集有 N 个数据点或实例，每个实例都是一个观测值或样本。

因此数据集 X 是一个 $N \times D$ 的矩阵，其中每一行代表一个实例，每一列代表一个特征。表示为：

$$\mathbf{X} = \begin{bmatrix} [\mathbf{x}^{(1)}]^\top \\ [\mathbf{x}^{(2)}]^\top \\ \vdots \\ [\mathbf{x}^{(N)}]^\top \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_D^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_D^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \cdots & x_D^{(N)} \end{bmatrix}$$

其中， $\mathbf{x}^{(n)}$ 是第 n 个实例的特征向量， $x_d^{(n)}$ 是第 n 个实例在第 d 个特征上的值。

多变量数据的均值 μ 是一个向量，表示每个特征的平均值。

数学表达式为： $\mu = \mathbb{E}[\mathbf{x}] = \begin{pmatrix} \mu_1 \\ \vdots \\ \mu_d \end{pmatrix}$

其中， μ_i 是第*i*个特征的均值。

协方差（Covariance）矩阵 Σ 描述了不同特征之间的协方差关系。

数学表达式为： $\Sigma = \text{Cov}(\mathbf{x}) = \mathbb{E}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^\top] = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1D} \\ \sigma_{12} & \sigma_2^2 & \cdots & \sigma_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{D1} & \sigma_{D2} & \cdots & \sigma_D^2 \end{pmatrix}$

其中， σ_i^2 是第*i*个特征的方差， σ_{ij} 是第*i*个和第*j*个特征之间的协方差。

统计量 μ 和 Σ 唯一定义了一个多变量高斯分布，记作 $\mathcal{N}(\mu, \Sigma)$ 或 $\mathcal{N}(\mathbf{x}; \mu, \Sigma)$ 。

需要注意：这并不适用于一般分布。

因此多变量高斯分布的概率密度函数（正态分布的随机变量 $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ 的分布）为：

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right]$$

其中：

$\mathbf{x}$ 是一个*d*-维随机向量。

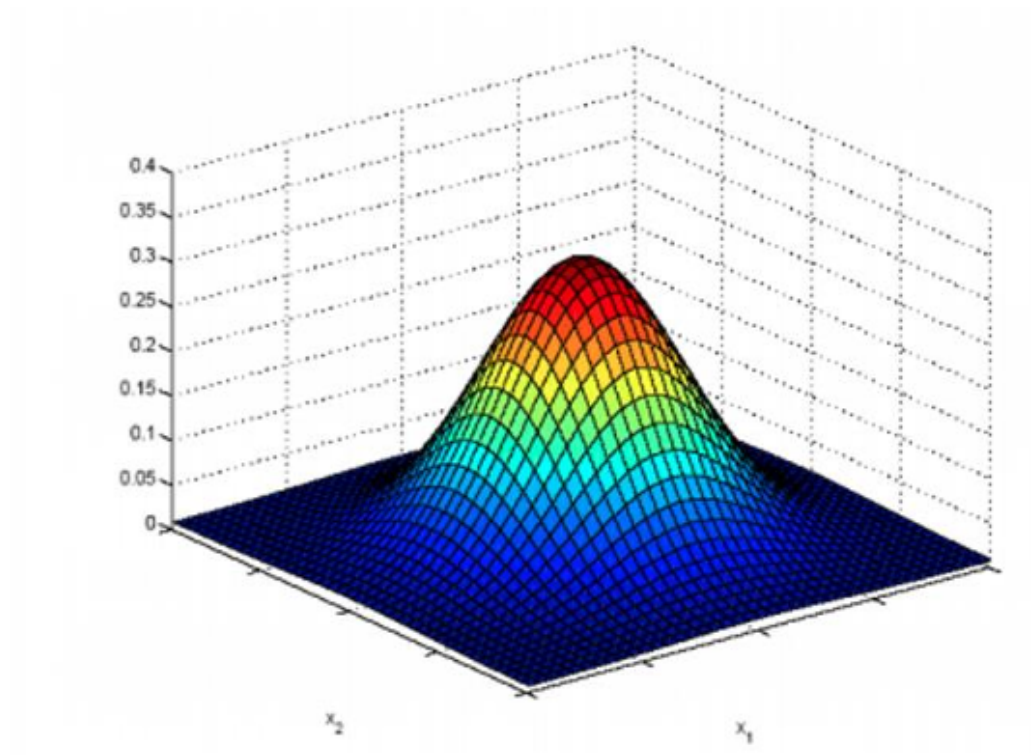
μ 是均值向量。

Σ 是协方差矩阵。

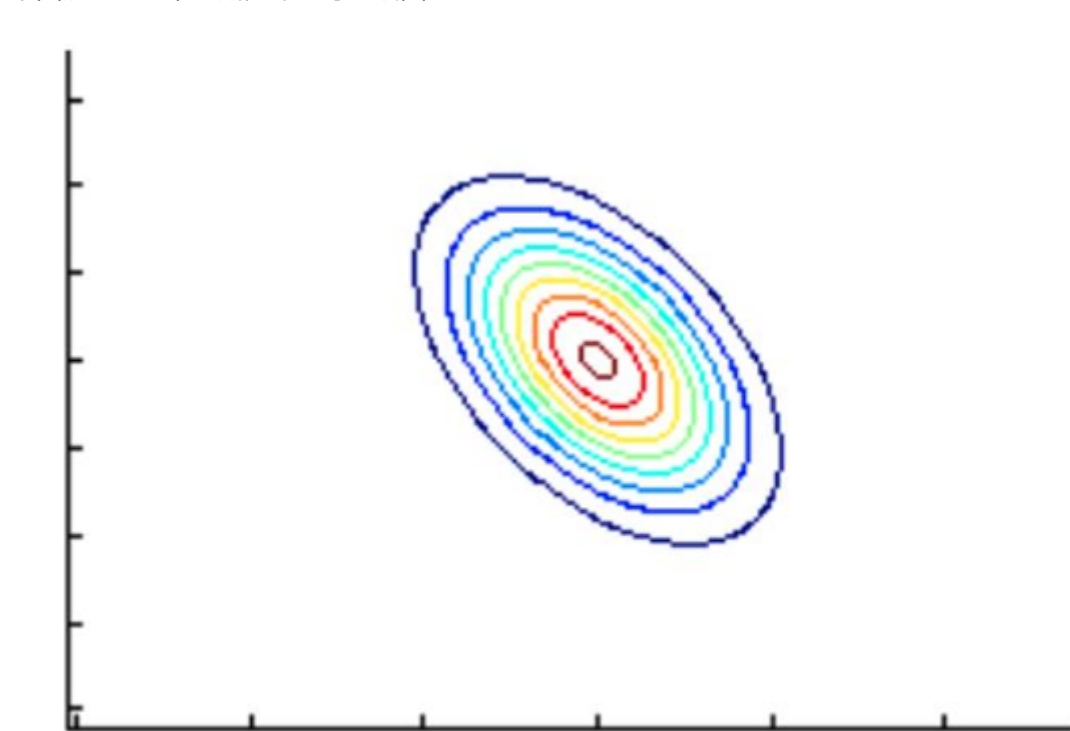
$|\Sigma|$ 是协方差矩阵的行列式。

Σ^{-1} 是协方差矩阵的逆矩阵。

下图展示了二维高斯分布的三维图。



下图展示了二维高斯分布的等高线图。



2.4.3 生成模型 (The Generative Model)

我们现在聚焦于生成模型。

生成模型是一种假设数据是由一系列潜在的生成过程产生的模型。在聚类问题中，生成模型可以帮助我们理解数据是如何被组织成不同的簇。

假设数据点 $\mathbf{x}$ 是按照以下步骤生成的：

1. 从 $\{1, \dots, K\}$ 中选择一个簇 z ，选择的概率为 $p(z = k) = \pi_k$ 。这里， π_k 表示数据点属于簇 k 的先验概率。

2. 给定簇 z ，从高斯分布 $\mathcal{N}(\mathbf{x}|\mu_z, \mathbf{I})$ 中采样数据点 $\mathbf{x}$ 。这里， μ_z 是簇 z 的均值向量， $\mathbf{I}$ 是单位协方差矩阵。

这个过程可以用以下数学表达式表示：

$$p(z = k) = \pi_k$$

$$p(\mathbf{x}|z = k) = \mathcal{N}(\mathbf{x}|\mu_k, \mathbf{I})$$

我们可以使用贝叶斯规则 (Bayes' rule) 来计算数据点属于某个簇的概率。

定义了联合分布 $p(z, \mathbf{x}) = p(z)p(\mathbf{x}|z)$ 其中 z 是簇标签， x 是数据点， $p(z)$ 是簇的先验概率， $p(x|z)$ 是给定簇标签时数据点的概率。这里的参数为 $\{\pi_k, \mu_k\}_{k=1}^K$ 。

数据点 x 的边缘分布由所有可能的簇标签 z 的联合分布的和给出： $p(\mathbf{x}) = \sum_z p(z, \mathbf{x})$ 。

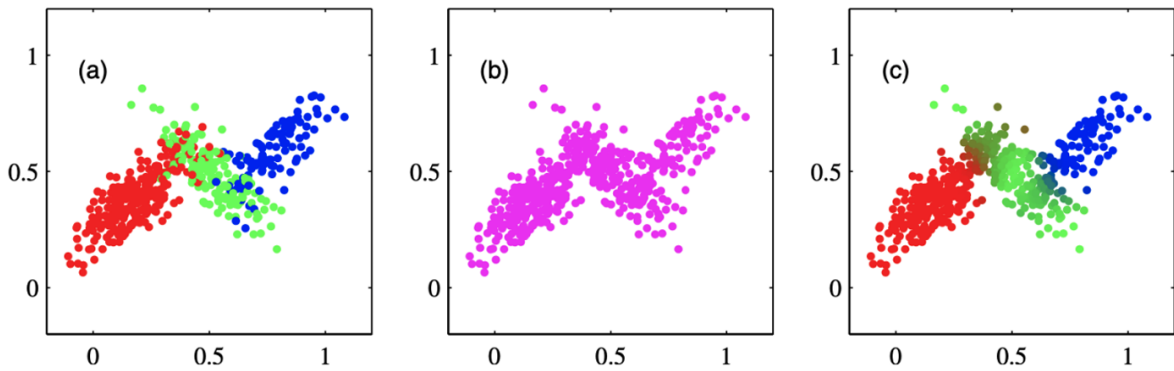
使用贝叶斯规则可以计算给定数据点 $\mathbf{x}$ 时，它属于簇 k 的概率 $p(z = k|\mathbf{x})$ ：

$$p(z = k|\mathbf{x}) = \frac{p(\mathbf{x}|z=k)p(z=k)}{p(\mathbf{x})}$$

这个公式告诉我们数据点 $\mathbf{x}$ 来自第 k 个簇的概率。

下图展示了一个生成模型的示例，是一个由三个高斯分布混合而成的模型。

这个示例中，数据集由500个点组成，这些点是从三个不同的高斯分布中抽取的。



a) Samples from $p(\mathbf{x} | z)$

b) Samples from the marginal $p(\mathbf{x})$

c) Responsibilities $p(z | \mathbf{x})$

图a：展示了从条件分布 $p(x|z)$ 中抽取的样本。这里，每个颜色代表一个不同的高斯分布（簇），显示了在给定簇标签 z 的情况下，数据点是如何分布的。

图b：展示了从边缘分布 $p(x)$ 中抽取的样本。边缘分布是所有簇的混合，显示了不考虑簇标签时数据点的整体分布。这通常会导致数据点的分布看起来更加混合和模糊。

图c：展示了数据点的责任度 $p(z|x)$ ，即给定数据点 x 时，它属于每个簇的概率。这个图可以帮助我们理解每个数据点最有可能属于哪个簇。

2.4.4 最大似然估计

那我们如何选择参数 $\{\pi_k, \mu_k\}_{k=1}^K$ 呢。

我们使用最大似然估计来解决这个问题。

最大似然原则是选择参数以最大化观测数据的似然性 (likelihood)。似然性是指在给定参数的情况下，观测到当前数据的概率。

在聚类问题中，簇分配 z 是潜在变量，我们并不直接观测到这些变量，只观测到数据 x 。

给定数据集 $\mathcal{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$ ，选择参数以最大化： $\log p(\mathcal{D}) = \sum_{n=1}^N \log p(\mathbf{x}^{(n)})$

通过边缘化 (marginalizing out) 潜在变量 z 来找到 $p(x)$ ：

$$p(\mathbf{x}) = \sum_{k=1}^K p(z = k, \mathbf{x}) = \sum_{k=1}^K p(z = k)p(\mathbf{x}|z = k)$$

这里， $p(z = k)$ 是簇 k 的先验概率， $p(x|z = k)$ 是给定簇 k 时数据点 x 的条件概率。

2.4.5 高斯混合模型 (GMM)

我们回到高斯混合模型上，数据点 x 的概率密度函数 $p(x)$ ：

$$p(\mathbf{x}) = \sum_{k=1}^K p(z = k)p(\mathbf{x}|z = k) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \mathbf{I})$$

其中：

K 是簇的数量。

π_k 是第 k 个簇的混合系数（先验概率）。

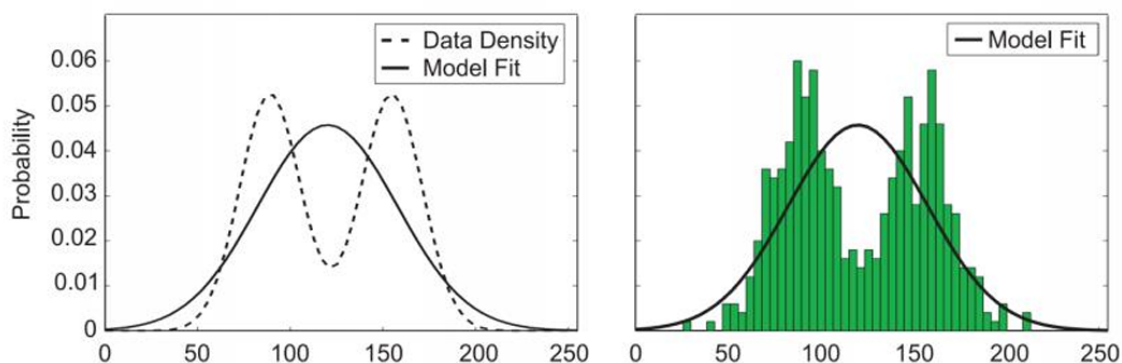
$\mathcal{N}(\mathbf{x}|\mu_k, \mathbf{I})$ 是均值为 μ_k 且协方差矩阵为单位矩阵 $\mathbf{I}$ 的高斯分布。

混合系数 π_k ：

π_k 表示数据点属于第 k 个簇的概率。

为了简化模型，我们假设每个簇的协方差矩阵 Σ_k 为单位矩阵 $\mathbf{I}$ 以简化模型。

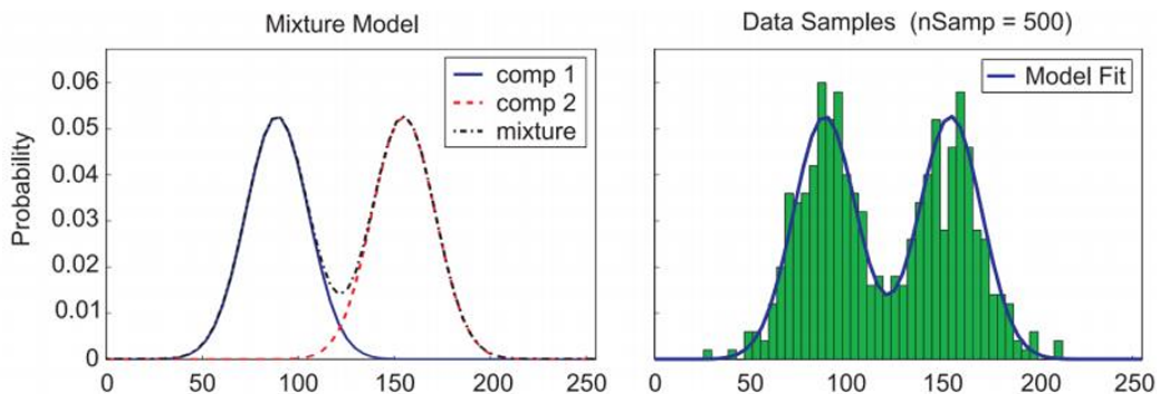
下图展示了一维高斯混合模型拟合数据的情况。



图中显示了数据的真实密度（虚线）和模型拟合的高斯分布（实线）。

可以看到，单个高斯分布无法很好地捕捉数据的双峰特性，拟合效果不佳。

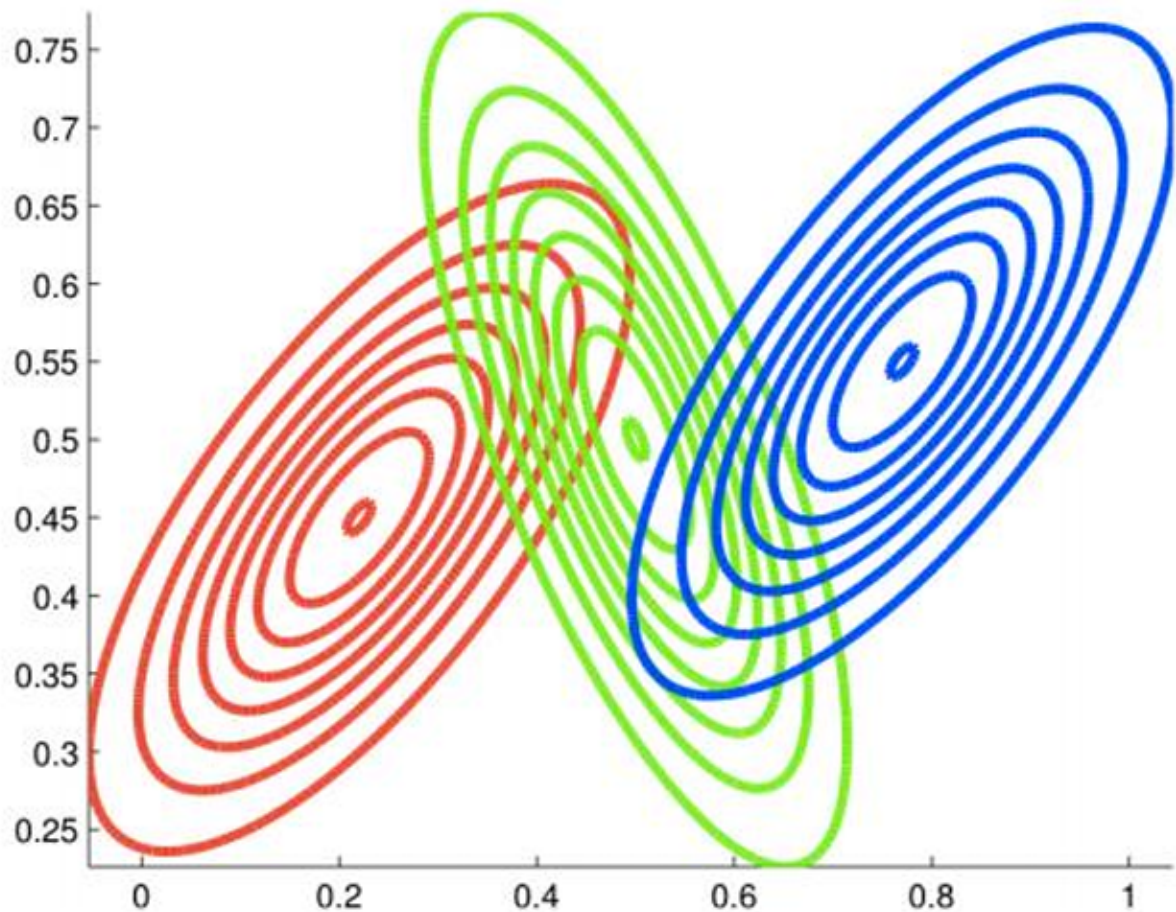
下图用两个高斯分布混合而成的模型来拟合数据，即 $K = 2$ 。



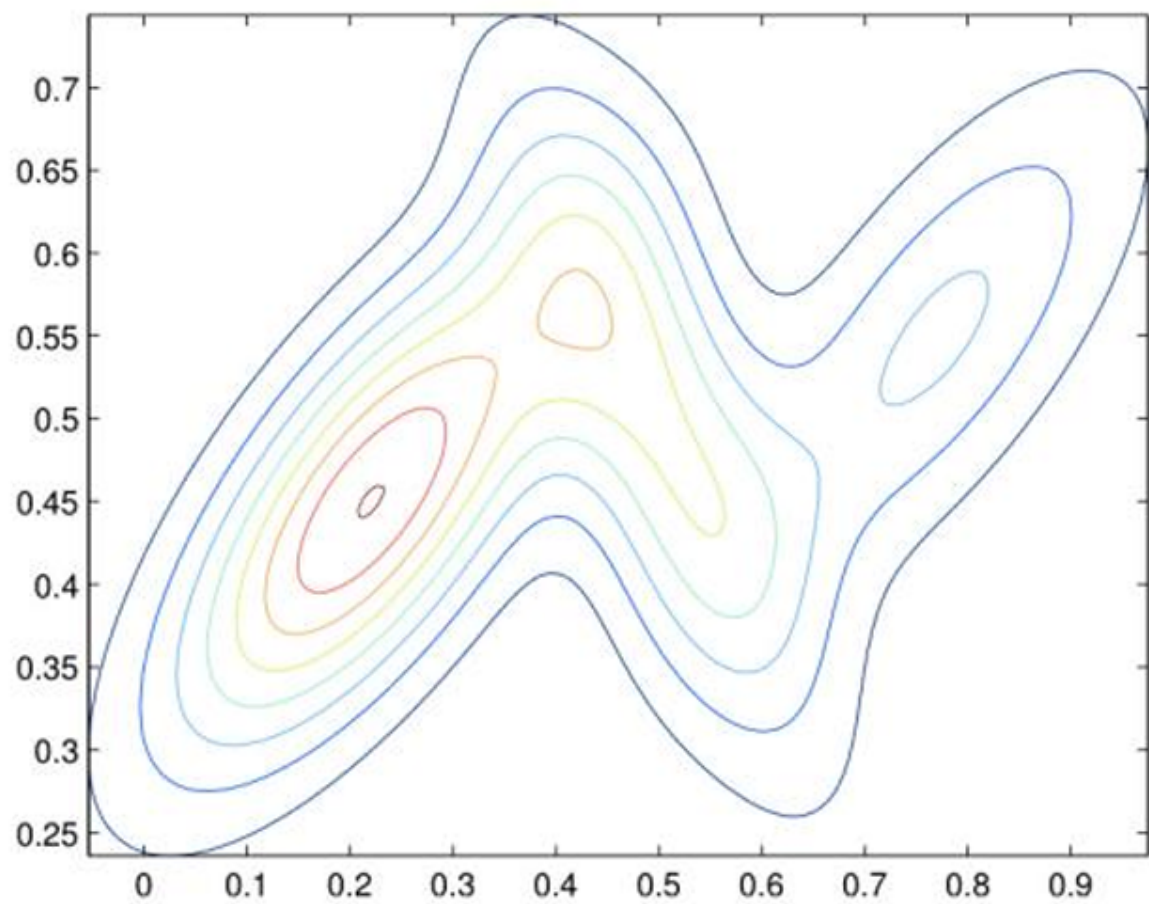
可以看到 GMM 模型更好地捕捉了数据的双峰特性。

下图展示了二维高斯分布（2D Gaussians）的混合模型。

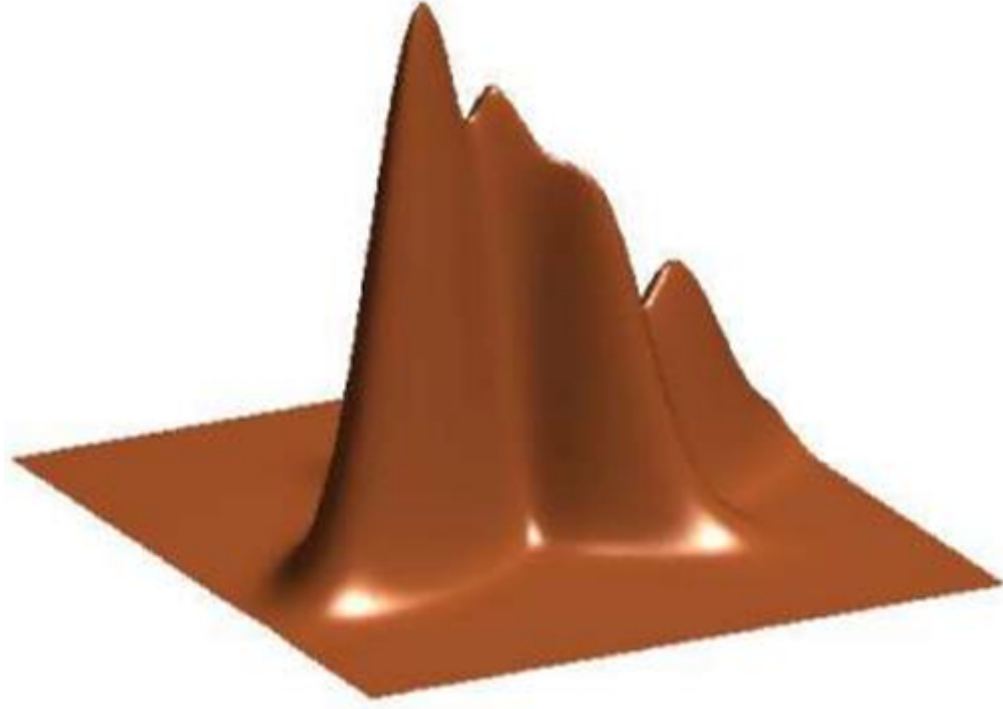
图中展示了多个二维高斯分布的等高线图，每个颜色代表一个不同的高斯分布（簇）。



下图展示了混合模型的等高线图，它显示了各个高斯分布的组合效果，其中每个高斯分布的贡献由其混合系数决定。



下图的三维图形展示了混合模型的概率密度函数，其中 z -轴表示概率密度。



2.4.5.1 使用最大似然估计 (Maximum Likelihood Estimation, MLE) 来拟合高斯混合模型 (Gaussian Mixture Model, GMM)

最大似然目标函数的表达式：

$$\log p(\mathcal{D}) = \sum_{n=1}^N \log p(\mathbf{x}^{(n)}) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I}) \right)$$

这里， $\mathcal{D}$ 是数据集， N 是数据点的数量， K 是高斯分布的数量， π_k 是第 k 个分布的混合系数， $\mathcal{N}(\mathbf{x} | \mu_k, \mathbf{I})$ 是均值为 μ_k 且协方差矩阵为单位矩阵 $\mathbf{I}$ 的高斯分布。

问题是如何优化这个目标函数相对于参数 $\{\pi_k, \mu_k\}$ 。

直接对参数求导并设为0没有封闭形式的解，因为这是对数函数内部的求和。

这使得优化过程变得困难。

一种可能的优化方法是梯度上升法 (Gradient Ascent)。

我们现在介绍使用最大似然估计 (Maximum Likelihood, ML) 在已知潜在变量 (latent variables) 的情况下如何简化问题。

我们观察可知，如果我们知道了每个 $\mathbf{x}^{(n)}$ 对应的 $z^{(n)}$ ，即我们的完整数据集是

$\mathcal{D}_{\text{complete}} = \{(z^{(n)}, \mathbf{x}^{(n)})\}_{n=1}^N$ ，那么最大似然问题就变得简单：

完整数据集的对数似然可以表示为：

$$\log p(\mathcal{D}_{\text{complete}}) = \sum_{n=1}^N \log p(z^{(n)}, \mathbf{x}^{(n)})$$

这可以进一步分解为：

$$= \sum_{n=1}^N \log p(\mathbf{x}^{(n)} | z^{(n)}) + \log p(z^{(n)})$$

其中， $p(\mathbf{x}^{(n)} | z^{(n)})$ 是给定簇标签时数据点的条件概率， $p(z^{(n)})$ 是簇标签的先验概率。

进一步，如果我们考虑每个簇 k 的贡献，可以写成：

$$= \sum_{n=1}^N \sum_{k=1}^K \mathbb{I}[z^{(n)} = k] (\log \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I}) + \log \pi_k)$$

这里， $\mathbb{I}[z^{(n)} = k]$ 是指示函数，当 $z^{(n)} = k$ 时为1，否则为0。 $\mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I})$ 是均值为 μ_k 且协方差矩阵为单位矩阵 $\mathbf{I}$ 的高斯分布。

给定完整数据集 $\mathcal{D}_{\text{complete}}$ ，对数似然函数可以表示为

$$\log p(\mathcal{D}_{\text{complete}}) = \sum_{n=1}^N \sum_{k=1}^K \mathbb{I}[z^{(n)} = k] (\log \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I}) + \log \pi_k)$$

这里， $\mathbb{I}[z^{(n)} = k]$ 是指示函数，当 $z^{(n)} = k$ 时为1，否则为0。 $\mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I})$ 是均值为 μ_k 且协方差矩阵为单位矩阵 $\mathbf{I}$ 的高斯分布。

通过最大化 $\log p(\mathcal{D}_{\text{complete}})$ ，我们可以得到簇均值 $\hat{\mu}_k$ 和混合系数 $\hat{\pi}_k$ 的估计：

$$\hat{\mu}_k = \frac{\sum_{n=1}^N \mathbb{I}[z^{(n)}=k] \mathbf{x}^{(n)}}{\sum_{n=1}^N \mathbb{I}[z^{(n)}=k]} = \text{class means}$$

$$\hat{\pi}_k = \frac{1}{N} \sum_{n=1}^N \mathbb{I}[z^{(n)} = k] = \text{class proportions}$$

这里， $\hat{\mu}_k$ 是第 k 个簇的均值， $\hat{\pi}_k$ 是第 k 个簇的混合系数（或先验概率）。

这种方法在实际应用中通常不适用，因为我们通常不知道簇标签。然而，它为理解最大似然估计提供了一个理论基础。

我们没有直接观察到簇分配 $z^{(n)}$ ，但可以使用贝叶斯规则来计算 $p(z^{(n)}|\mathbf{x}^{(n)})$ ，即给定数据点 $\mathbf{x}^{(n)}$ 时它属于簇 $z^{(n)}$ 的概率。

使用贝叶斯规则，给定 $\mathbf{x}$ 的条件下 z 的条件概率可以表示为： $p(z = k|\mathbf{x}) = \frac{p(z=k)p(\mathbf{x}|z=k)}{p(\mathbf{x})}$

这里， $p(z = k)$ 是簇 k 的先验概率， $p(\mathbf{x}|z = k)$ 是给定簇 k 时数据点 $\mathbf{x}$ 的条件概率， $p(\mathbf{x})$ 是数据点 $\mathbf{x}$ 的边缘概率。

边缘概率 $p(\mathbf{x})$ 可以通过对所有可能的簇标签求和来计算：

$$p(\mathbf{x}) = \sum_{j=1}^K p(z = j)p(\mathbf{x}|z = j)$$

在高斯混合模型中，条件概率 $p(\mathbf{x}|z = k)$ 通常是一个高斯分布：

$$p(\mathbf{x}|z = k) = \mathcal{N}(\mathbf{x}|\mu_k, \mathbf{I})$$

因此，条件概率 $p(z = k|\mathbf{x})$ 可以进一步表示为：

$$p(z = k|\mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x}|\mu_k, \mathbf{I})}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \mathbf{I})}$$

这里 $\mathcal{N}(\mathbf{x}|\mu_k, \mathbf{I})$ 是均值为 μ_k 且协方差矩阵为单位矩阵 $\mathbf{I}$ 的高斯分布。

所以给定完整数据集 $\mathcal{D}_{\text{complete}}$ ，对数似然函数可以表示为：

$$\log p(\mathcal{D}_{\text{complete}}) = \sum_{n=1}^N \sum_{k=1}^K \mathbb{I}[z^{(n)} = k] (\log \mathcal{N}(\mathbf{x}^{(n)}|\mu_k, \mathbf{I}) + \log \pi_k)$$

我们不知道簇分配 $\mathbb{I}[z^{(n)} = k]$ ，但我们知道它们的期望 $\mathbb{E}[\mathbb{I}[z^{(n)} = k]|\mathbf{x}^{(n)}] = p(z^{(n)} = k|\mathbf{x}^{(n)})$ ：

$$\mathbb{E}[\mathbb{I}[z^{(n)} = k]|\mathbf{x}^{(n)}] = p(z^{(n)} = k|\mathbf{x}^{(n)})$$

如果我们将 $r_k^{(n)} = p(z^{(n)} = k|\mathbf{x}^{(n)})$ 代入 $\mathbb{I}[z^{(n)} = k]$ ，我们可以得到：

$$\sum_{n=1}^N \sum_{k=1}^K r_k^{(n)} (\log \mathcal{N}(\mathbf{x}^{(n)}|\mu_k, \mathbf{I}) + \log \pi_k)$$

$$\text{这仍然很容易优化！解决方案与我们之前看到的类似：} \hat{\mu}_k = \frac{\sum_{n=1}^N r_k^{(n)} \mathbf{x}^{(n)}}{\sum_{n=1}^N r_k^{(n)}} \quad \hat{\pi}_k = \frac{\sum_{n=1}^N r_k^{(n)}}{N}$$

$$\text{注意：这只在我们把} r_k^{(n)} \text{视为固定时才有效，即：} r_k^{(n)} = \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)}|\mu_k, \mathbf{I})}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(n)}|\mu_j, \mathbf{I})}$$

2.4.5.2 EM算法

因此我们使用EM算法（期望最大化算法）来拟合一个高斯分布的混合。

EM算法是一种迭代方法，用于在存在潜在变量的情况下估计模型参数。它包括两个主要步骤：期望步骤（E-step）和最大化步骤（M-step）。

1. E-step（期望步骤）：

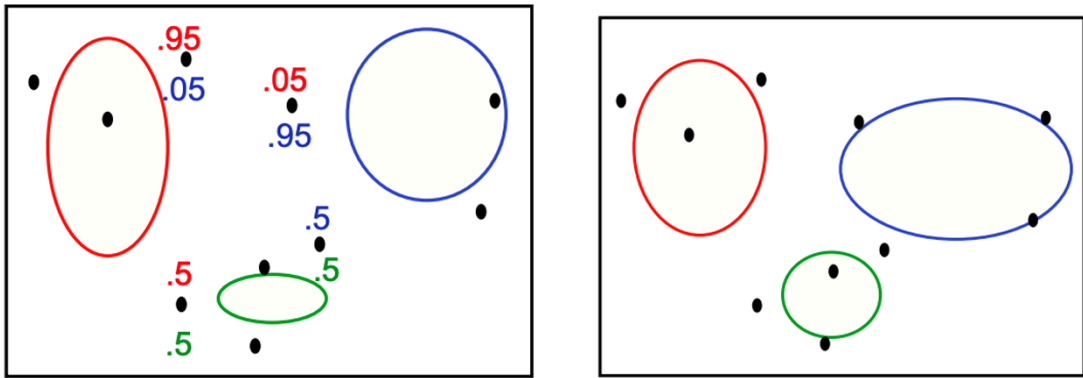
在当前模型的基础上计算后验概率 $r_k^{(n)} = p(z^{(n)} = k|\mathbf{x}^{(n)})$ ，即每个簇对生成一个数据点的责任度。

这表示我们认为每个簇对生成特定数据点的贡献程度。

2. M-step（最大化步骤）：

使用上一步计算的后验概率 $r_k^{(n)}$ 来更新参数，假设 $r_k^{(n)}$ 是固定的。

也就是改变每个高斯分布的参数，以最大化它生成当前负责的数据的概率。



以这样的方式，簇参数被调整以更好地拟合数据。

2.4.5.2.1 EM算法的数学表达

1. 初始化:

初始化均值 $\hat{\mu}_k$ 和混合系数 $\hat{\pi}_k$ 。

2. 迭代直到收敛:

算法在两个主要步骤之间交替进行，直到收敛。

• E-step (期望步骤):

给定当前参数，计算每个数据点 $\mathbf{x}^{(n)}$ 属于每个簇 k 的责任度 $r_k^{(n)}$:

$$r_k^{(n)} = p(z^{(n)} = k | \mathbf{x}^{(n)}) = \frac{\hat{\pi}_k \mathcal{N}(\mathbf{x}^{(n)} | \hat{\mu}_k, \mathbf{I})}{\sum_{j=1}^K \hat{\pi}_j \mathcal{N}(\mathbf{x}^{(n)} | \hat{\mu}_j, \mathbf{I})}$$

• M-step (最大化步骤):

给定当前的责任度，重新估计参数: $\hat{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N r_k^{(n)} \mathbf{x}^{(n)}$

$$\hat{\pi}_k = \frac{N_k}{N} \quad \text{with} \quad N_k = \sum_{n=1}^N r_k^{(n)}$$

这里， N_k 是分配给簇 k 的数据点的数量。

我们可以计算对数似然函数并检查算法是否收敛:

$$\log p(\mathcal{D}) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \hat{\pi}_k \mathcal{N}(\mathbf{x}^{(n)} | \hat{\mu}_k, \mathbf{I}) \right)$$

2.4.5.2.2 EM算法的小结

直接优化最大似然目标 $\sum_{n=1}^N \log p(\mathbf{x}^{(n)})$ 是困难的，因为它涉及到对数函数内部的求和，这使得直接求导数并设为0来找到最优解变得不可能。

如果我们有完整的数据集 $\mathcal{D}_{\text{complete}}$ ，即我们知道每个数据点 $\mathbf{x}^{(n)}$ 对应的簇标签 $z^{(n)}$ ，那么完整数据的似然目标就变得容易优化:

$$\sum_{n=1}^N \log p(z^{(n)}, \mathbf{x}^{(n)}) = \sum_{n=1}^N \sum_{k=1}^K \mathbb{I}[z^{(n)} = k] (\log \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I}) + \log \pi_k)$$

这里， $\mathbb{I}[z^{(n)} = k]$ 是指示函数，当 $z^{(n)} = k$ 时为1，否则为0。 $\mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I})$ 是均值为 μ_k 且协方差矩阵为单位矩阵 $\mathbf{I}$ 的高斯分布。

我们不知道 $z^{(n)}$ 它们是潜在变量)，所以我们用责任度 $r_k^{(n)} = p(z^{(n)} = k | \mathbf{x}^{(n)})$ 来替换 $\mathbb{I}[z^{(n)} = k]$ 。

也就是说：我们用 $\mathbb{I}[z^{(n)} = k]$ 在 $p(z^{(n)} | \mathbf{x}^{(n)})$ 下的期望来替换它 (E-step)。

我们最终得到了期望完整数据对数似然 (Expected Complete Data Log-likelihood):

$$\sum_{n=1}^N \mathbb{E}_{p(z^{(n)} | \mathbf{x}^{(n)})} [\log p(z^{(n)}, \mathbf{x}^{(n)})] = \sum_{n=1}^N \sum_{k=1}^K r_k^{(n)} (\log \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \mathbf{I}) + \log \pi_k)$$

其中 $\mathbb{E}_{p(z^{(n)} | \mathbf{x}^{(n)})}$ 表示在给定当前模型参数 $\{\pi_k, \mu_k\}$ 下关于 $z^{(n)}$ 的期望。

$r_k^{(n)} = p(z^{(n)} = k | \mathbf{x}^{(n)})$ 是数据点 $\mathbf{x}^{(n)}$ 属于簇 k 的责任度。

我们在参数 $\{\pi_k, \mu_k\}_k$ 上最大化上述期望完整数据对数似然。

EM算法在两个步骤之间交替进行:

- E-step (期望步骤): 计算责任度 $r_k^{(n)} = p(z^{(n)} = k | \mathbf{x}^{(n)})$ ，即给定当前模型参数 π_k, μ_k 下的期望 $\mathbb{E}[\mathbb{I}[z^{(n)} = k] | \mathbf{x}^{(n)}]$
- M-step (最大化步骤): 用新得到的 $\{r_k^{(n)}\}$ 更新模型参数 π_k, μ_k ，使期望完整数据对数似然最大化。

2.4.5.2.3 K-Means 和 EM 算法 的对比

K-Means 步骤:

1. 分配步 (Assignment)
把每个点 硬分配到最近的簇中心
2. 重拟合步 (Refitting)
把簇中心移到被分配点的均值

EM 算法步骤:

1. E-step
计算每个点对每个簇的后验概率 (软分配)
2. M-step
用当前后验概率加权, 最大化期望似然, 更新簇参数 (均值、权重等)